

Algorytmy Metaheurystyczne, Lista 1

Błażej Pawluk

07.04.2026

1 Rozwiązania zadań

1.1 Średnia wartość uzyskanego rozwiązania

dataset	1. losowa permutacja	2. najlepsze losowe sąsiedztwo	3. MST
wi29	28271	42582.3	28173.2
dj38	6904	11605.3	6926.86
qa194	10152.9	16947.2	9912.43
uy734	86931.1	174026	84422.1
zi929	104192	155496	101955
mu1978	94191.7	182995	93405.7
ca4663	-	-	1387930
tz6117	-	-	430944

Tabela 1: Porównanie średniej wartości uzyskanego rozwiązania

Wyniki najbliższe optymalnego rozwiązania osiągnął algorytm wykorzystujący minimalne drzewo rozpinające, a najgorzej poradził sobie algorytm wybierający najlepsze spośród losowo wybranych sąsiedztw.

1.2 Średnia liczba kroków poprawy

dataset	1. losowa permutacja	2. najlepsze losowe sąsiedztwo	3. MST
wi29	23.3103	39.4828	10.3333
dj38	35.1316	48.1579	8.71429
qa194	212.742	363.778	51.3571
uy734	923.503	1590.26	166.464
zi929	1175.16	2425.57	218.774
mu1978	2791.89	5093.83	408.444
ca4663	-	-	982.217
tz6117	-	-	1496.91

Tabela 2: Porównanie liczby kroków potrzebnych do uzyskania rozwiązania

Najmniej kroków poprawy, a co za tym idzie najszybciej działa algorytm wykorzystujący minimalne drzewo rozpinające, a najgorzej algorytm wybierający najlepsze spośród losowo wybranych sąsiedztw, co jednak się nie przekłada już na czas działania algorytmu - obliczenia wykonują się o wiele szybciej, gdy trzeba policzyć o rząd mniej kosztów w każdym kroku.

1.3 Rozmiar minimalnego drzewa rozpinającego

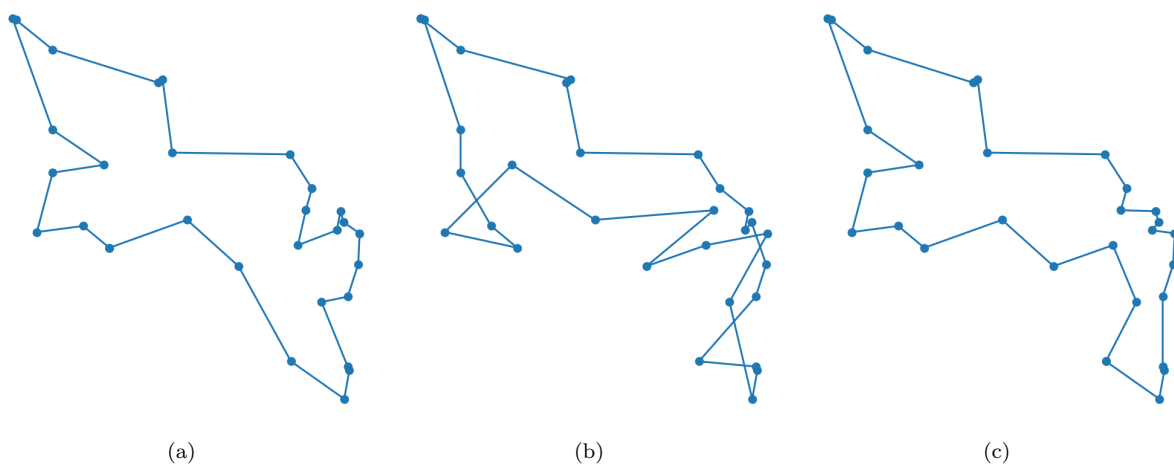
dataset	MST	znaleziony cykl
wi29	21750.6	28173.2
dj38	5830.86	6926.86
qa194	8028.01	9912.43
uy734	69876.5	84422.1
zi929	82872.6	101955
mu1978	74099.9	93405.7
ca4663	111856e0	1387930
tz6117	348001	430944

Tabela 3: Porównanie średniej wartości uzyskanego rozwiązania i rozmiaru minimalnego drzewa rozpinającego

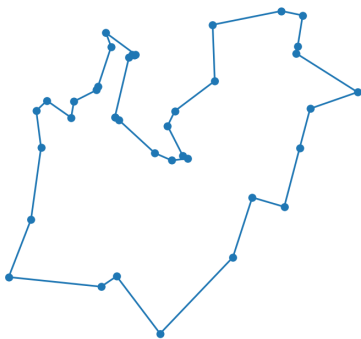
1.4 Najlepsze znalezione rozwiązania

(a) - losowa permutacja, (b) - najlepsze losowe sąsiedztwo, (c) - MST

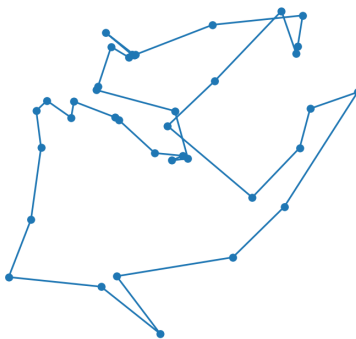
1.4.1 wi29 - Western Sahara



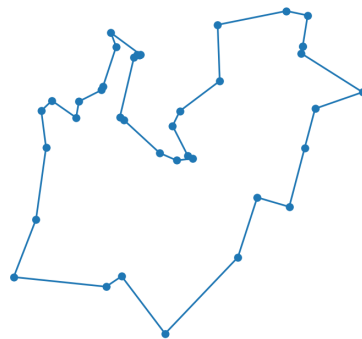
1.4.2 dj38 - Djibouti



(d)

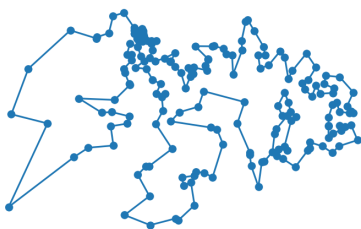


(e)

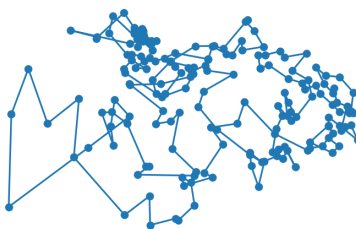


(f)

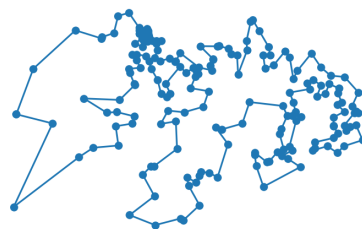
1.4.3 qa194 - Qatar



(g)



(h)

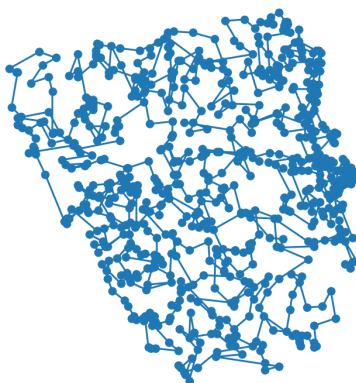


(i)

1.4.4 uy734 - Uruguay



(j)

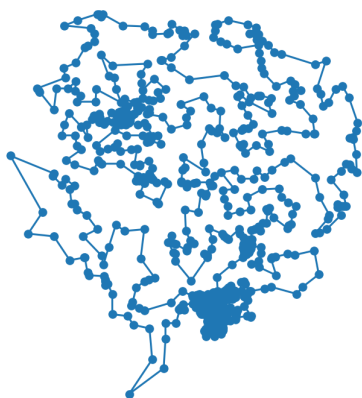


(k)

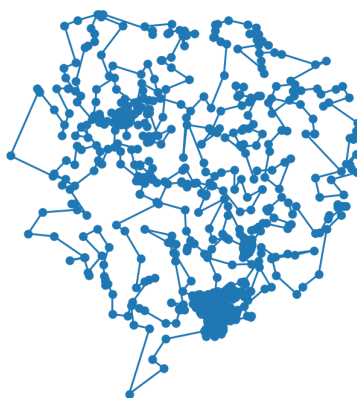


(l)

1.4.5 zi929 - Zimbabwe



(m)



(n)

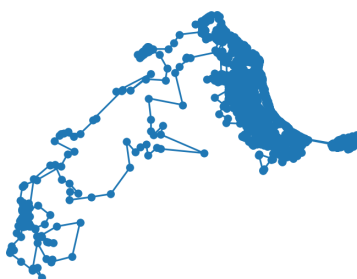


(o)

1.4.6 mu1979 - Oman



(p)

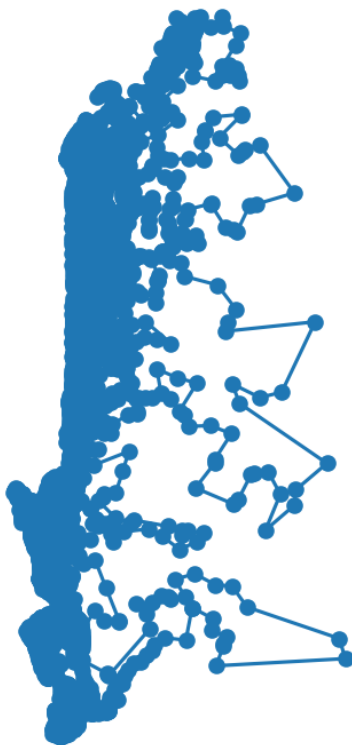


(q)



(r)

1.4.7 ca4663 - Canada



(s)

1.4.8 tz6117 - Tanzania



(t)

2 Odpowiedzi na dodatkowe pytania

2.1 Jak duże jest otoczenie jednej permutacji dla n wierzchołków?

Rozmiar otoczenia jednej permutacji dla n wierzchołków to liczba wszystkich par (i, j) , gdzie $i < j$, czyli $\binom{n}{2}$.

2.2 Dlaczego algorytm Local Search dla grafów Euklidesowych zawsze zwróci rozwiązanie bez krzyżujących się krawędzi?

Założmy, że mamy 2 przecinające się krawędzie: $(A, B), (C, D)$. W geometrii Euklidesowej z nierówności trójkąta wiemy, że zachodzi nierówność: $|AB| + |CD| > |AC| + |BD|$, zatem zamiana usuwa przecięcie i zmniejsza długość trasy. Gdyby po wykonanym algorytmie Local Search w grafie wynikowym były przecinające się krawędzie, to oznaczałoby, że można ulepszyć lokalnie rozwiązanie ruchem invert - algorytm by się nie skończył, co daje sprzeczność.

2.3 Kontrprzykład dla powyższego pytania, gdy używamy zaokrągleń.

Weźmy prostokąt o bokach o długości 1 i 2. Wtedy cykl bez przecinających się krawędzi ma długość $1 + 2 + 1 + 2 = 6$, natomiast algorytm może też znaleźć cykl złożony z przekątnych (o długości $\sqrt{1^2 + 2^2} = \sqrt{5}$, co daje koszt krawędzi 2) i krótszych krawędzi o koszcie $1 + 2 + 1 + 2 = 6$. Pozbycie się tego cyklu nie ulepszyłoby rozwiązania, więc algorytm by zostawił jako wynik cykl z przecinającymi się krawędziami.